Throwing Exceptions

CS300: Programming II

Cole Nelson

Student Org Shoutout

MadHacks and more...

Learning Objectives

1. Be able to **distinguish** `throw` from `throws`.
2. Be able to **trace** an exception up the call stack.
3. Be able to **construct** a user-defined exception class.
4. Be able to **write** a tester method that passes only when a method throws.

Eclipse Exercise

Download today's starter from Canvas and code along.

Same `Person` class as Wednesday -- today the age check gets an exception of its own.

Refusing bad input

```
if (age < 1 || age > 100) {
  throw new IllegalArgumentException("age must be 1-100, got " + age);
}
```

`new` builds the object. `throw` is what stops everything — no return value, no next line.

Step through it in Java Visualizer

Which one is which?

```
public static Person register(String name, int age) throws InvalidAgeException {
  if (age < 1 || age > 100) {
    throw new InvalidAgeException(name + " gave an age of " + age);
  }
  return new Person(name, age);
}
```

One of these two words does something. The other makes a promise. Which is which – and what breaks if you delete each one?

`throw` and `throws` are different words

	Where it goes	What it does
`throw`	in the method body	raises one, right now
`throws`	in the signature line	warns that this method might

`throws` signals a warning

```
public static Person register(String name, int age) throws InvalidAgeException
```

This does **not** throw anything. It says *"callers, be ready"* – and for a checked exception, the compiler will not let you call it until you are.

The caller? Either handles it, or it too throws it up.

Exceptions pop frames until they're caught

An (uncaught) exception abandons the current method, then asks its caller, then *its* caller, all the way up until it is caught.

If nobody catches it, `main` pops too — and the JVM prints the path it climbed.

Trace it in Java Visualizer

A trace is a travel log

```
java.lang.NumberFormatException: For input string: "twenty-one"
        at java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)
        at java.lang.Integer.parseInt(Integer.java:580)
        at java.lang.Integer.parseInt(Integer.java:615)
        at Main.parseAge(Main.java:7) // <-- your code starts here
        at Main.parseAgeNextYear(Main.java:11)
        at Main.main(Main.java:17)
```

The frames above yours are inside Java's own library. Scan down to the first `Main` and start there.

Tophat Q1

A check on where an exception ends up.

→ Open Tophat to vote.

Eclipse Exercise

Complete TODO 1.

When the built-in exceptions don't fit

`IllegalArgumentException` covers a bad number. What covers *"nobody is 240 years old"*?

Nothing built in. So write the class — an exception is just an object, and you already know how to make those. They depend on a class, next week's topic.

A user-defined exception

```
public class InvalidAgeException extends IllegalArgumentException {
  public InvalidAgeException(String message) {
    super(message);
  }
}
```

Two lines of real content. `extends IllegalArgumentException` makes it **unchecked**; `extends Exception` would make it **checked**.

We cover `extends` properly in Week 5 — today, just the pattern.

Eclipse Exercise

Complete TODO 2.

Wednesday you closed Scanner by hand

```
} finally {
  scanner.close();          // or the file handle leaks
}
```

A `Scanner` holds an open resource. Every path out of that method had to remember to close it.

Declare it in the parentheses instead

```
try (Scanner in = new Scanner(new File(roster))) {
  while (in.hasNextLine()) { System.out.println(in.nextLine()); }
} catch (FileNotFoundException e) {
  System.out.println("no such file: " + roster);
}
```

No `finally`. Java closes it on the way out — no matter what. Also, a `FileNotFoundException` is **checked**; the compiler will not let you ignore it.

Run it in Java Visualizer

Eclipse Exercise

Complete TODO 3.

Testing exceptions

```
public static boolean testRejectsInvalidAge() {
  try {
    Main.register("Bucky", 240);
    return false;           // it came back, so it did NOT throw
  } catch (IllegalArgumentException e) {
    return true;            // exactly what we wanted
  }
}
```

To test an exception throwing method, we prove that it `throws` for invalid input (and doesn't for valid input)!

Run the tester in Java Visualizer

Tophat Q2

A check on testing exceptions.

→ Open Tophat to vote.

Questions?